

```
In [1]: import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: import os
import time

from tqdm import tqdm

import numpy as np
import pandas as pd

import hyperopt.hp as hp

from sklearn.model_selection import train_test_split

from catboost import CatBoostClassifier

from sklift.datasets import fetch_x5
from sklift.models import ClassTransformation, TwoModels
from sklift.metrics import qini_auc_score, uplift_at_k

from causalml.inference.tree import UpliftTreeClassifier as UpliftTreeClassi

from upninja.dml.uplift_tree_dml import UpliftTreeRegressorDML
from upninja.tune.selection import UpliftTune

import matplotlib.pyplot as plt
import seaborn as sns
```

Failed to import duecredit due to No module named 'duecredit'

```
In [3]: SEED = 8
```

```
In [4]: %%time
```

```
dataset = fetch_x5()
dataset.data.keys()
```

CPU times: user 26.9 s, sys: 5.94 s, total: 32.8 s
Wall time: 35.3 s

```
Out[4]: dict_keys(['clients', 'train', 'purchases'])
```

```
In [5]: %%time
```

```
print(f'Dataset type: {type(dataset)}\n')
print(f'Dataset features shape: {dataset.data['clients'].shape}')
print(f'Dataset features shape: {dataset.data['train'].shape}')
print(f'Dataset target shape: {dataset.target.shape}')
print(f'Dataset treatment shape: {dataset.treatment.shape}')
```

Dataset type: <class 'sklearn.utils._bunch.Bunch'>

Dataset features shape: (400162, 5)

Dataset features shape: (200039, 1)

Dataset target shape: (200039,)

Dataset treatment shape: (200039,)

CPU times: user 72 µs, sys: 88 µs, total: 160 µs

Wall time: 157 µs

In [6]: `%%time`

```
# Извлечение данных
df_clients = dataset.data['clients'].set_index('client_id')
df_train = pd.concat([dataset.data['train'], dataset.treatment, dataset.target], axis=1)
indices_test = pd.Index(set(df_clients.index) - set(df_train.index))

# Извлечение признаков
df_features = df_clients.copy()
df_features['first_issue_time'] = \
    (pd.to_datetime(df_features['first_issue_date'])
     - pd.Timestamp('1970-01-01')) // pd.Timedelta('1s')
df_features['first_redeem_time'] = \
    (pd.to_datetime(df_features['first_redeem_date'])
     - pd.Timestamp('1970-01-01')) // pd.Timedelta('1s')
df_features['issue_redeem_delay'] = df_features['first_redeem_time'] \
    - df_features['first_issue_time']
df_features = df_features.drop(['first_issue_date', 'first_redeem_date'], axis=1)

indices_learn, indices_valid = train_test_split(df_train.index, test_size=0.2)
```

CPU times: user 186 ms, sys: 38.8 ms, total: 225 ms

Wall time: 224 ms

In [7]: `%%time`

```
X_train = df_features.loc[indices_learn, :]
y_train = df_train.loc[indices_learn, 'target']
treat_train = df_train.loc[indices_learn, 'treatment_flg']

X_val = df_features.loc[indices_valid, :]
y_val = df_train.loc[indices_valid, 'target']
treat_val = df_train.loc[indices_valid, 'treatment_flg']

X_train_full = df_features.loc[df_train.index, :]
y_train_full = df_train.loc[:, 'target']
treat_train_full = df_train.loc[:, 'treatment_flg']

X_test = df_features.loc[indices_test, :]

X_train['gender'] = X_train['gender'].map({'F': 0, 'U': -1, 'M': 1})
X_val['gender'] = X_val['gender'].map({'F': 0, 'U': -1, 'M': 1})
X_test['gender'] = X_test['gender'].map({'F': 0, 'U': -1, 'M': 1})

X_train.fillna(-1.0, inplace=True)
X_val.fillna(-1.0, inplace=True)
X_test.fillna(-1.0, inplace=True)
```

```
cat_features = ['gender']
```

CPU times: user 250 ms, sys: 5.51 ms, total: 256 ms

Wall time: 255 ms

✓ Train models

★ Class Transformation

In [8]: %%time

```
cb_params = cb_hp_space = {
    'iterations': hp.uniformint('iterations', 100, 2500),
    'depth': hp.uniformint('depth', 2, 10),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2),
    'l2_leaf_reg': hp.uniform('l2_leaf_reg', 1, 10),
    'verbose': False,
    'thread_count': -1,
    'random_state': SEED
}

cb_tune = UpliftTune(
    base_model_class=CatBoostClassifier,
    uplift_model_class=ClassTransformation,
    data=X_train,
    target=y_train,
    treatment=treat_train,
    space=cb_params,
    verbose=True,
    max_evals=10
)

t_time_start = time.process_time_ns()
cb_tuned = cb_tune.tune()['best_params']
t_time_end = time.process_time_ns()
```

100%|██████████| 10/10 [02:51<00:00, 17.13s/trial, best loss: -0.04954288228464684]

Optimization completed. Best score: 0.0495

Best parameters: {'depth': 2, 'iterations': 2166, 'l2_leaf_reg': 6.474242459540229, 'learning_rate': 0.06708005741019658, 'random_state': 8, 'thread_count': -1, 'verbose': False}

CPU times: user 20min 21s, sys: 2min 29s, total: 22min 51s

Wall time: 2min 51s

In [9]: %%time

```
ct = ClassTransformation(CatBoostClassifier(**cb_tuned))
f_time_start = time.process_time_ns()
ct = ct.fit(X_train, y_train, treat_train, estimator_fit_params={'cat_features': cat_features})
f_time_end = time.process_time_ns()
```

```
ct_uplift = ct.predict(X_val)
```

CPU times: user 3min, sys: 10.9 s, total: 3min 11s
Wall time: 21.4 s

★ DML Tree

```
In [10]: %%time

space = {
    'min_samples': hp.uniformint('min_samples', 20, 500),
    'max_depth': hp.uniformint('max_depth', 3, 20),
    'random_state': SEED
}

dml_tune = UpliftTune(
    uplift_model_class=UpliftTreeRegressorDML,
    data=X_train,
    target=y_train,
    treatment=treat_train,
    space=space,
    verbose=True,
    max_evals=10
)

t_time_start = time.process_time_ns()
dml_tuned = dml_tune.tune()['best_params']
t_time_end = time.process_time_ns()
```

100%|██████████| 10/10 [00:09<00:00, 1.07trial/s, best loss: -0.05889112202442145]

Optimization completed. Best score: 0.0589

Best parameters: {'max_depth': 14, 'min_samples': 411, 'random_state': 8}

CPU times: user 9.05 s, sys: 169 ms, total: 9.21 s

Wall time: 9.33 s

```
In [11]: %%time

dml = UpliftTreeRegressorDML(**dml_tuned)

f_time_start = time.process_time_ns()
dml = dml.fit(X_train, y_train, treat_train)
f_time_end = time.process_time_ns()

dml_uplift = dml.predict(X_val)
```

CPU times: user 539 ms, sys: 7.53 ms, total: 547 ms

Wall time: 550 ms

✓ Case

Оценим эффективность назначения коммуникаций при помощи uplift-моделей.

```

In [12]: def build_no_horizon_business_case(
    base_df,
    uplift_col,
    quantiles=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0),
    sms_cost=8,
    check_col='avg_purchase_sum'
):
    """
    Бизнес-кейс без горизонтов:
    - сортируем клиентов по uplift-score;
    - Top N% считаются накопительно сверху скоринга;
    - конверсия считается по исходному target Retail Hero;
    - денежная оценка использует исторический средний чек как проху;
    - ОП на клиента = uplift конверсии * средний чек - стоимость SMS;
    - Total increment = ОП на клиента * N клиентов.
    """

    df = base_df.copy()

    df_sorted = (
        df
        .sort_values(by=uplift_col, ascending=False)
        .reset_index(drop=True)
    )

    total_n = len(df_sorted)
    rows = []

    for q in quantiles:
        cutoff = int(total_n * q)
        subset = df_sorted.iloc[:cutoff].copy()

        treat = subset[subset['treatment_flg'] == 1]
        control = subset[subset['treatment_flg'] == 0]

        conv_treat = treat['target'].mean()
        conv_control = control['target'].mean()
        uplift_conv = conv_treat - conv_control

        # Средний чек как проху стоимости покупки
        avg_check = subset.loc[subset['target'] == 1, check_col].mean()

        if pd.isna(avg_check):
            avg_check = subset[check_col].mean()

        op_per_client = uplift_conv * avg_check - sms_cost
        total_increment = op_per_client * len(subset)

        rows.append({
            'Модель': uplift_col,
            'Доля базы': f'Top {int(q * 100)}%',
            'Доля базы sort': int(q * 100),
            'Кол-во наблюдений': len(subset),

            'Конверсия в КГ': conv_control,

```

```

        'Конверсия в ТГ': conv_treat,
        'Инкремент конверсии': uplift_conv,

        'Средний чек проху, руб': avg_check,

        'Инкремент ОП на клиента, руб': op_per_client,
        'Total increment, руб': total_increment,
    })

    return pd.DataFrame(rows)

```

```

In [13]: def make_no_horizon_slide_table(long_df):
    df = long_df.copy()

    df['Конверсия в КГ'] = (
        (df['Конверсия в КГ'] * 100)
        .round(1)
        .astype(str)
        .str.replace('.', ',', regex=False)
        + '%'
    )

    df['Конверсия в ТГ'] = (
        (df['Конверсия в ТГ'] * 100)
        .round(1)
        .astype(str)
        .str.replace('.', ',', regex=False)
        + '%'
        + df['Инкремент конверсии'].apply(
            lambda x: f"({x * 100:.1f}%) " if x >= 0 else f"({x * 100:.1f}%) "
        ).str.replace('.', ',', regex=False)
    )

    df['Средний чек проху, руб'] = df['Средний чек проху, руб'].round(1)
    df['Инкремент ОП на клиента, руб'] = df['Инкремент ОП на клиента, руб'].round(0)
    df['Total increment, руб'] = df['Total increment, руб'].round(0).astype(int)

    final = df[[
        'Доля базы',
        'Кол-во наблюдений',
        'Конверсия в КГ',
        'Конверсия в ТГ',
        'Средний чек проху, руб',
        'Инкремент ОП на клиента, руб',
        'Total increment, руб'
    ]].copy()

    final['_sort'] = final['Доля базы'].str.extract(r'(\d+)').astype(int)
    final = final.sort_values('_sort').drop(columns='_sort').reset_index(drop=True)

    return final

```

```

In [14]: def get_best_strategy(case_df):
    strategy = case_df.copy()

```

```

strategy = strategy[
    strategy['Инкремент ОП на клиента, руб'] > 0
].copy()

best = (
    strategy
    .sort_values('Total increment, руб', ascending=False)
    .head(1)
    .copy()
)

best['Конверсия в КГ'] = (best['Конверсия в КГ'] * 100).round(1)
best['Конверсия в ТГ'] = (best['Конверсия в ТГ'] * 100).round(1)
best['Инкремент конверсии'] = (best['Инкремент конверсии'] * 100).round(1)
best['Средний чек проху, руб'] = best['Средний чек проху, руб'].round(1)
best['Инкремент ОП на клиента, руб'] = best['Инкремент ОП на клиента, руб'].round(1)
best['Total increment, руб'] = best['Total increment, руб'].round(0).astype(int)

return best[[
    'Модель',
    'Доля базы',
    'Кол-во наблюдений',
    'Конверсия в КГ',
    'Конверсия в ТГ',
    'Инкремент конверсии',
    'Средний чек проху, руб',
    'Инкремент ОП на клиента, руб',
    'Total increment, руб'
]]

```

```

In [15]: def sensitivity_by_sms_cost(base_case_df, sms_costs=(3, 5, 8, 10, 15)):
    rows = []

    for cost in sms_costs:
        tmp = base_case_df.copy()

        tmp['Инкремент ОП на клиента, руб'] = (
            tmp['Инкремент конверсии'] * tmp['Средний чек проху, руб'] - cost
        )
        tmp['Total increment, руб'] = (
            tmp['Инкремент ОП на клиента, руб'] * tmp['Кол-во наблюдений']
        )

        best = (
            tmp[tmp['Инкремент ОП на клиента, руб'] > 0]
            .sort_values('Total increment, руб', ascending=False)
            .head(1)
        )

        if len(best) > 0:
            rows.append({
                'Стоимость SMS, руб': cost,
                'Оптимальная доля базы': best['Доля базы'].iloc[0],
                'Кол-во наблюдений': best['Кол-во наблюдений'].iloc[0],
                'ОП на клиента, руб': round(best['Инкремент ОП на клиента, руб'], 1),
                'Total increment, руб': round(best['Total increment, руб'], 1)
            })

```

```
)
```

```
return pd.DataFrame(rows)
```

```
In [16]: def budget_strategy(base_df, case_df, budgets=(50_000, 100_000, 200_000, 500_000),
rows = []
total_base = len(base_df)

for budget in budgets:
    n_clients = int(budget // sms_cost)
    share = n_clients / total_base

    # ищем ближайший доступный Top N%
    tmp = case_df.copy()
    tmp['share'] = tmp['Доля базы'].str.extract(r'(\d+)').astype(int) /
    tmp['diff'] = (tmp['share'] - share).abs()

    row = tmp.sort_values('diff').head(1).iloc[0]

    rows.append({
        'Бюджет, руб': budget,
        'Макс. клиентов по бюджету': n_clients,
        'Ближайшая доля базы': row['Доля базы'],
        'ОП на клиента, руб': round(row['Инкремент ОП на клиента, руб'],
        'Ожидаемый total increment, руб': round(row['Total increment, руб'],

    })

return pd.DataFrame(rows)
```

```
In [17]: X_val['ct_uplift'] = ct_uplift
X_val['dml_uplift'] = dml_uplift
X_val = pd.concat([X_val, treat_val, y_val], axis=1)
```

```
In [18]: X_val = X_val.merge(dataset.data['purchases'][['client_id', 'purchase_sum',
```

```
In [19]: base_val = df_features.loc[indices_valid, :].copy().reset_index()

if 'client_id' not in base_val.columns:
    base_val = base_val.rename(columns={base_val.columns[0]: 'client_id'})

base_val['ct_uplift'] = ct_uplift
base_val['dml_uplift'] = dml_uplift

base_val = base_val.merge(
    treat_val.rename('treatment_flg').reset_index(),
    on='client_id',
    how='left'
)

base_val = base_val.merge(
    y_val.rename('target').reset_index(),
    on='client_id',
    how='left'
)
```

```
base_val = base_val.drop_duplicates(subset=['client_id']).copy()

print(base_val.shape)
```

(60012, 10)

```
In [20]: # Исторический средний чек по клиенту
# purchases – это история покупок, используем ее только как проху для денег

purchase_proxy = (
    dataset.data['purchases']
    .groupby('client_id')
    .agg(
        avg_purchase_sum=('purchase_sum', 'mean'),
        total_purchase_sum=('purchase_sum', 'sum'),
        purchase_cnt=('purchase_sum', 'count')
    )
    .reset_index()
)

base_val = base_val.merge(
    purchase_proxy,
    on='client_id',
    how='left'
)

base_val['avg_purchase_sum'] = base_val['avg_purchase_sum'].fillna(
    base_val['avg_purchase_sum'].median()
)

base_val['total_purchase_sum'] = base_val['total_purchase_sum'].fillna(0)
base_val['purchase_cnt'] = base_val['purchase_cnt'].fillna(0)

base_val.head()
```

```
Out[20]:
```

	client_id	age	gender	first_issue_time	first_redeem_time	issue_redeem_delay
0	e2f4387414	45	M	1519842117	1.525813e+09	5971259.0
1	8098ef0f15	18	F	1515614436	NaN	NaN
2	79c9d3a1a8	59	F	1499945782	1.501761e+09	1814782.0
3	a299331961	34	U	1507227952	1.514053e+09	6825460.0
4	f5df2493fa	60	F	1499508384	1.509122e+09	9613778.0

```
In [28]: dml_case = build_no_horizon_business_case(
    base_df=base_val,
    uplift_col='dml_uplift',
    quantiles=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0),
    sms_cost=8,
    check_col='avg_purchase_sum'
)

dml_slide_table = make_no_horizon_slide_table(dml_case)
```

dml_slide_table

Out [28]:

	Доля базы	Кол-во наблюдений	Конверсия в КГ	Конверсия в ТГ	Средний чек проху, руб	Инкремент ОП на клиента, руб	Total increment, руб
0	Top 10%	6001	58,9%	68,8% (+9,8%)	544.9	45.6	273611
1	Top 20%	12002	62,0%	68,7% (+6,7%)	541.1	28.5	341728
2	Top 30%	18003	61,6%	67,3% (+5,7%)	557.3	23.8	428931
3	Top 40%	24004	61,4%	66,7% (+5,3%)	580.6	22.7	545009
4	Top 50%	30006	61,7%	66,9% (+5,2%)	602.0	23.2	694810
5	Top 60%	36007	61,9%	66,9% (+5,0%)	618.7	22.6	815002
6	Top 70%	42008	61,8%	66,6% (+4,7%)	621.4	21.4	898166
7	Top 80%	48009	61,3%	65,5% (+4,1%)	619.6	17.7	850186
8	Top 90%	54010	61,2%	65,0% (+3,8%)	626.7	16.0	862777
9	Top 100%	60012	60,0%	63,7% (+3,7%)	626.1	15.2	910562

```
In [29]: ct_case = build_no_horizon_business_case(
    base_df=base_val,
    uplift_col='ct_uplift',
    quantiles=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0),
    sms_cost=8,
    check_col='avg_purchase_sum'
)

ct_slide_table = make_no_horizon_slide_table(ct_case)

ct_slide_table
```

Out [29]:

	Доля базы	Кол-во наблюдений	Конверсия в КГ	Конверсия в ТГ	Средний чек проху, руб	Инкремент ОП на клиента, руб	Total increment, руб
0	Top 10%	6001	64,6%	74,5% (+9,9%)	519.0	43.4	260173
1	Top 20%	12002	63,4%	70,9% (+7,4%)	547.5	32.8	393163
2	Top 30%	18003	63,0%	69,7% (+6,7%)	571.9	30.1	542414
3	Top 40%	24004	62,6%	68,4% (+5,9%)	588.3	26.5	634920
4	Top 50%	30006	62,5%	67,8% (+5,3%)	603.2	24.2	726044
5	Top 60%	36007	62,2%	67,2% (+5,1%)	612.4	23.0	828401
6	Top 70%	42008	61,7%	66,5% (+4,8%)	619.2	21.7	910950
7	Top 80%	48009	61,5%	65,7% (+4,3%)	625.6	18.7	895986
8	Top 90%	54010	60,7%	64,8% (+4,0%)	628.8	17.4	939972
9	Top 100%	60012	60,0%	63,7% (+3,7%)	626.1	15.2	910562

In [30]:

```
dml_best_strategy = get_best_strategy(dml_case)
ct_best_strategy = get_best_strategy(ct_case)

display(dml_best_strategy)
display(ct_best_strategy)
```

	Модель	Доля базы	Кол-во наблюдений	Конверсия в КГ	Конверсия в ТГ	Инкремент конверсии	Средний чек проху, руб	Инкремент клиента
9	dml_uplift	Top 100%	60012	60.0	63.7	3.7	626.1	
8	ct_uplift	Top 90%	54010	60.7	64.8	4.0	628.8	

```
In [24]: dml_sensitivity = sensitivity_by_sms_cost(dml_case)
ct_sensitivity = sensitivity_by_sms_cost(ct_case)

dml_sensitivity
```

Out [24]:

	Стоимость SMS, руб	Оптимальная доля базы	Кол-во наблюдений	ОП на клиента, руб	Total increment, руб
0	3	Top 100%	60012	20.17	1210622.0
1	5	Top 100%	60012	18.17	1090598.0
2	8	Top 100%	60012	15.17	910562.0
3	10	Top 70%	42008	19.38	814150.0
4	15	Top 70%	42008	14.38	604110.0

```
In [25]: ct_sensitivity
```

Out [25]:

	Стоимость SMS, руб	Оптимальная доля базы	Кол-во наблюдений	ОП на клиента, руб	Total increment, руб
0	3	Top 100%	60012	20.17	1210622.0
1	5	Top 90%	54010	20.40	1102002.0
2	8	Top 90%	54010	17.40	939972.0
3	10	Top 90%	54010	15.40	831952.0
4	15	Top 70%	42008	14.69	616894.0

```
In [26]: dml_budget_strategy = budget_strategy(base_val, dml_case, sms_cost=8)
dml_budget_strategy
```

Out [26]:

	Бюджет, руб	Макс. клиентов по бюджету	Ближайшая доля базы	ОП на клиента, руб	Ожидаемый total increment, руб
0	50000	6250	Top 10%	48.59	291614.0
1	100000	12500	Top 20%	31.47	377734.0
2	200000	25000	Top 40%	25.70	617021.0
3	500000	62500	Top 100%	18.17	1090598.0

```
In [27]: ct_budget_strategy = budget_strategy(base_val, ct_case, sms_cost=8)
ct_budget_strategy
```

Out [27]:

	Бюджет, руб	Макс. клиентов по бюджету	Ближайшая доля базы	ОП на клиента, руб	Ожидаемый total increment, руб
0	50000	6250	Тор 10%	46.35	278176.0
1	100000	12500	Тор 20%	35.76	429169.0
2	200000	25000	Тор 40%	29.45	706932.0
3	500000	62500	Тор 100%	18.17	1090598.0